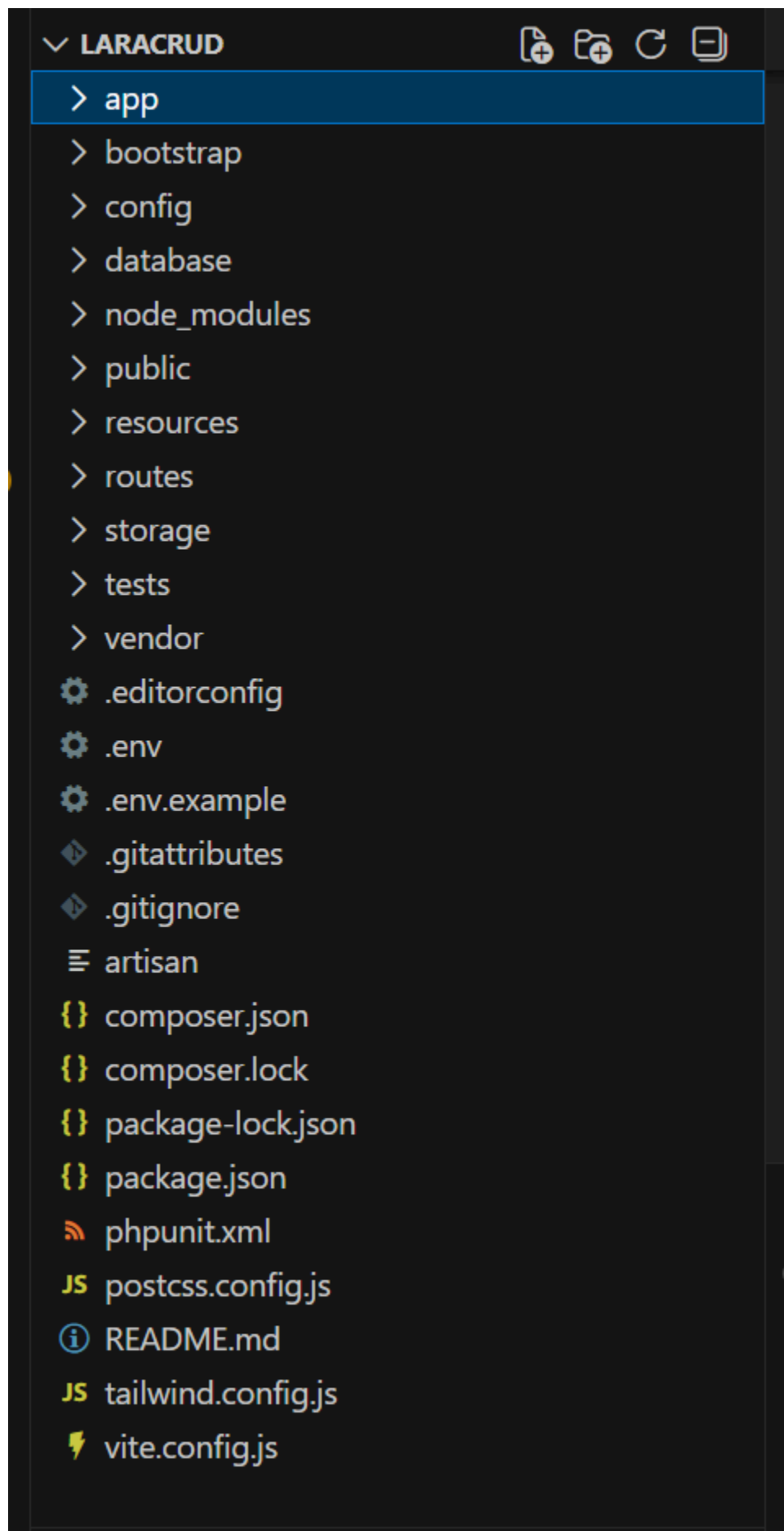

1.0 CREATE LARAVEL PROJECT

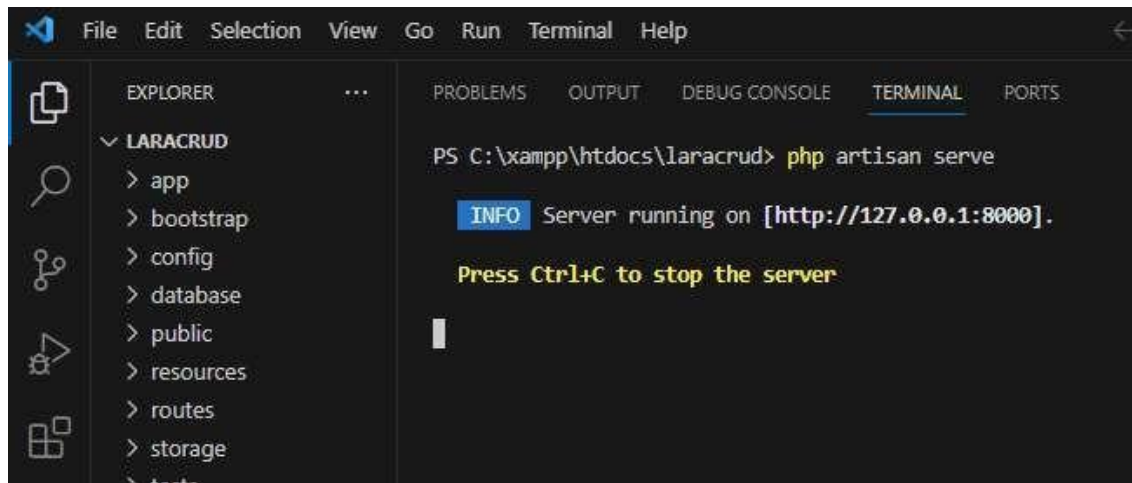
Run terminal from Visual Studio Code and create new Laravel project.

```
PS C:\> cd xampp
PS C:\xampp> cd htdocs
PS C:\xampp\htdocs> composer create-project laravel/laravel laracrud
```

Once successful, open project folder in Visual Studio Code



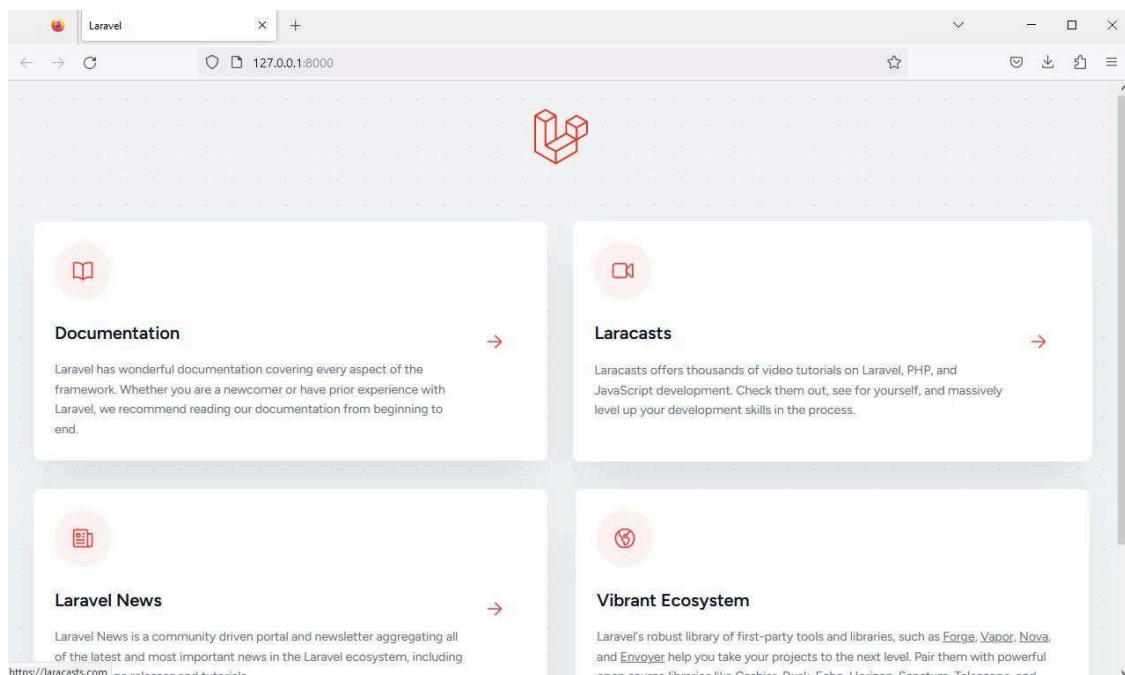
From terminal, start Laravel application and server.



The screenshot shows the Visual Studio Code interface with the 'TERMINAL' tab active. The terminal window displays the command `PS C:\xampp\htdocs\laracrud> php artisan serve`. Below the command, an 'INFO' message states 'Server running on [http://127.0.0.1:8000]'. A subsequent message in yellow text says 'Press Ctrl+C to stop the server'. The Explorer sidebar on the left shows the project structure for 'LARACRUD', including directories like 'app', 'bootstrap', 'config', 'database', 'public', 'resources', 'routes', 'storage', and 'tests'.

```
File Edit Selection View Go Run Terminal Help
EXPLORER
LARACRUD
  > app
  > bootstrap
  > config
  > database
  > public
  > resources
  > routes
  > storage
  > tests
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\xampp\htdocs\laracrud> php artisan serve
INFO Server running on [http://127.0.0.1:8000].
Press Ctrl+C to stop the server
```

View created application in web browser.



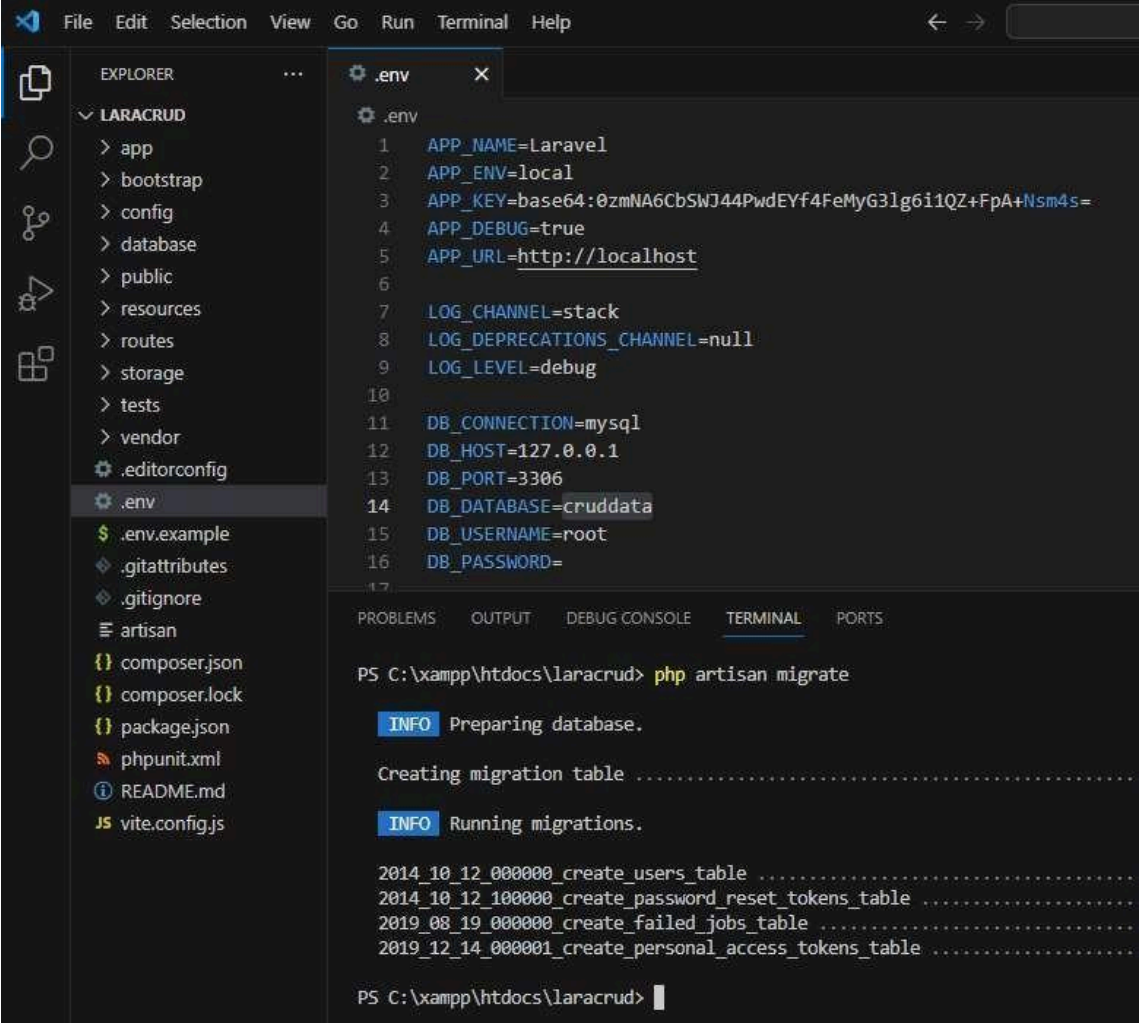
2.0 DATABASE

Run XAMPP and start Apache web server and MySQL RDBMS

Create a database in phpMyAdmin.



Migrate default tables to the created database.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows the project structure for 'LARACRUD', with the '.env' file selected. The main editor displays the contents of the '.env' file, which includes configuration for the application name, environment, key, debug mode, URL, logging, and database connection. The database connection is set to 'mysql' with host '127.0.0.1', port '3306', and database 'cruddata'. The terminal at the bottom shows the command 'php artisan migrate' being executed. The output indicates that the database is being prepared, migration tables are being created, and migrations are being run. The terminal output lists four migration tables being created with their respective timestamps and IDs.

```
.env
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:0zmNA6CbSWJ44PwdeYf4FeMyG3lg6i1QZ+FpA+Nsm4s=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 LOG_CHANNEL=stack
8 LOG_DEPRECATIONS_CHANNEL=null
9 LOG_LEVEL=debug
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=cruddata
15 DB_USERNAME=root
16 DB_PASSWORD=

PS C:\xampp\htdocs\laracrud> php artisan migrate

INFO Preparing database.

Creating migration table .....

INFO Running migrations.

2014_10_12_000000_create_users_table .....
2014_10_12_100000_create_password_reset_tokens_table .....
2019_08_19_000000_create_failed_jobs_table .....
2019_12_14_000001_create_personal_access_tokens_table .....

PS C:\xampp\htdocs\laracrud>
```

View the migrated tables at phpMyAdmin.

Server: 127.0.0.1 » Database: cruddata

Structure

SQL

Search

Query

Export

Import

Operations

Privileges

Routines

Events

Triggers

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ cache	★	4	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ cache_locks	★	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ failed_jobs	★	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ jobs	★	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ job_batches	★	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ migrations	★	3	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ password_reset_tokens	★	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ sessions	★	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ users	★	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
9 tables	Sum	12	InnoDB	utf8mb4_general_ci	224.0 KiB	0 B

View the User (table) model.

The screenshot shows an IDE with the following components:

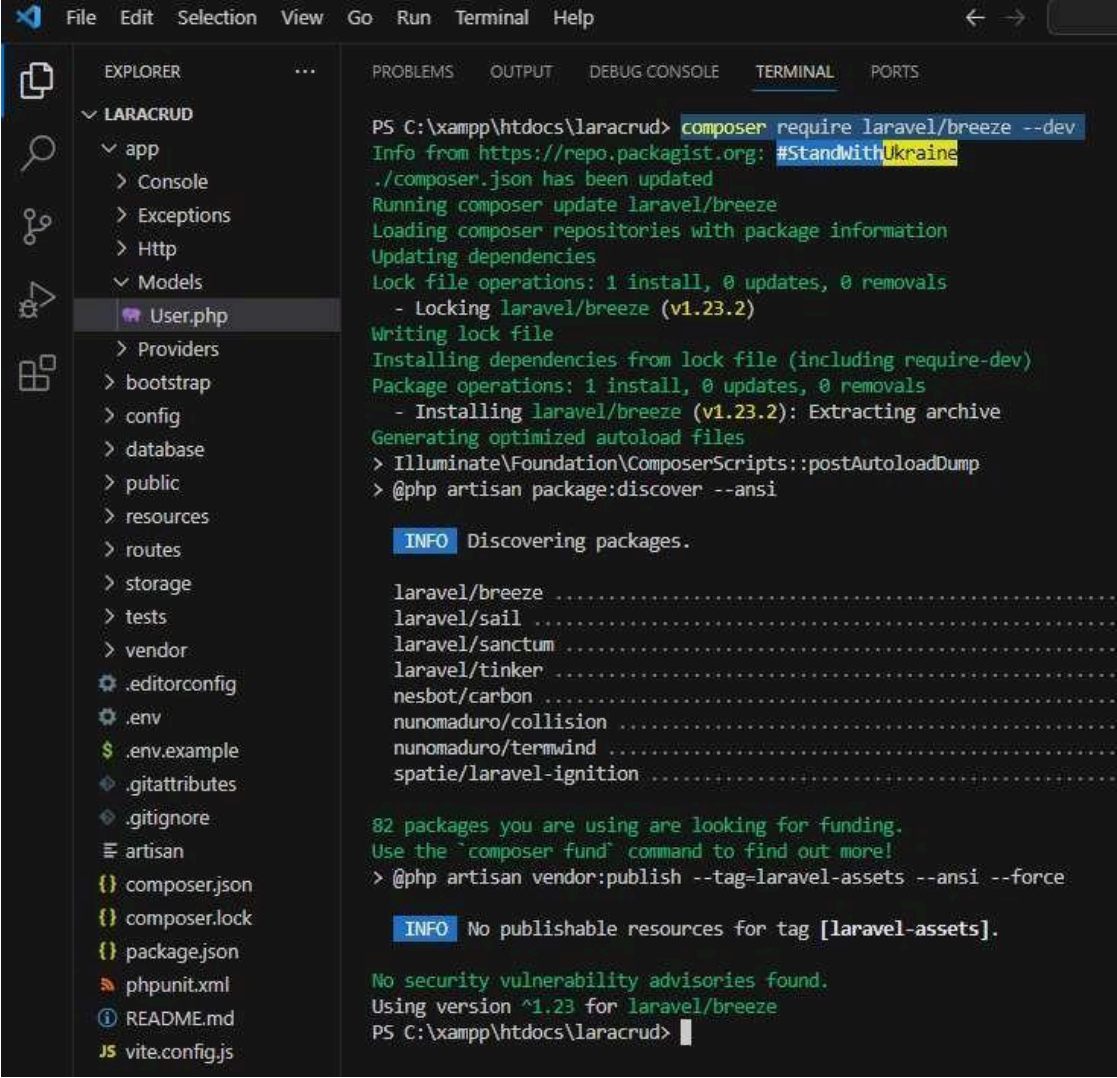
- EXPLORER:** A sidebar on the left showing the project structure. The 'Models' folder is expanded, and 'User.php' is selected.
- Code Editor:** The main area displays the contents of 'User.php'. The code is as follows:

```
1 <?php
2
3 namespace App\Models;
4
5 // use Illuminate\Contracts\Auth\MustVerifyEmail;
6 use Illuminate\Database\Eloquent\Factories\HasFactory;
7 use Illuminate\Foundation\Auth\User as Authenticatable;
8 use Illuminate\Notifications\Notifiable;
9
10 class User extends Authenticatable
11 {
12     /** @use HasFactory<\Database\Factories\UserFactory> */
13     use HasFactory, Notifiable;
14
15     /**
16      * The attributes that are mass assignable.
17      *
18      * @var list<string>
19      */
20     protected $fillable = [
21         'name',
22         'email',
23         'password',
24     ];
25 }
```
- TERMINAL:** A terminal window at the bottom shows the command prompt 'PS C:\xampp\htdocs\laracrud>'.

3.0 AUTHENTICATION (USING LARAVEL BREEZE)

Read the documentation for Laravel Breeze at <https://laravel.com/docs/10.x/starter-kits#laravel-breeze>

Install Laravel Breeze package from terminal.



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying the file structure of a project named 'LARACRUD'. The 'app' directory is expanded, showing files like 'Console', 'Exceptions', 'Http', 'Models', 'Providers', 'bootstrap', 'config', 'database', 'public', 'resources', 'routes', 'storage', 'tests', 'vendor', and configuration files like '.editorconfig', '.env', '.env.example', '.gitattributes', '.gitignore', 'artisan', 'composer.json', 'composer.lock', 'package.json', 'phpunit.xml', 'README.md', and 'vite.config.js'. The 'User.php' file is selected under 'Models'. The Terminal panel on the right shows the execution of the command 'composer require laravel/breeze --dev'. The output includes information from the Packagist repository, updates to the composer.json file, running the composer update command, loading repositories, updating dependencies, and locking the file operations. It shows that Laravel Breeze (v1.23.2) is being installed along with its dependencies. The terminal also displays a message about funding for 82 packages and a notice that no security vulnerability advisories were found. The command prompt is now at 'PS C:\xampp\htdocs\laracrud>'.

```
PS C:\xampp\htdocs\laracrud> composer require laravel/breeze --dev
Info from https://repo.packagist.org: #StandWithUkraine
./composer.json has been updated
Running composer update laravel/breeze
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
- Locking laravel/breeze (v1.23.2)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
- Installing laravel/breeze (v1.23.2): Extracting archive
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

INFO Discovering packages.

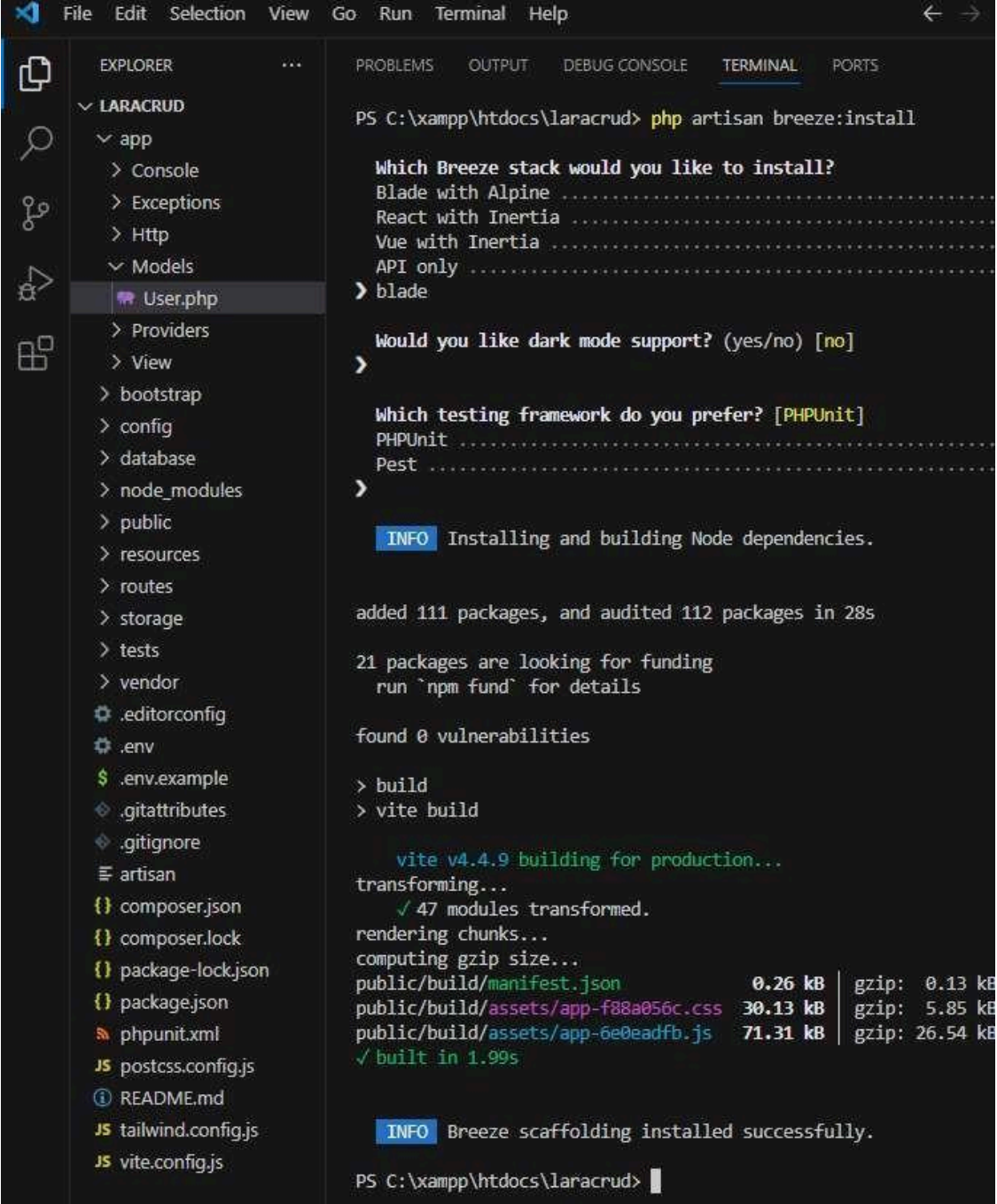
laravel/breeze .....
laravel/sail .....
laravel/sanctum .....
laravel/tinker .....
nesbot/carbon .....
nunomaduro/collision .....
nunomaduro/termwind .....
spatie/laravel-ignition .....

82 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
Using version ^1.23 for laravel/breeze
PS C:\xampp\htdocs\laracrud>
```


Install Breeze (Blade), Node.js resources.



The screenshot shows the Visual Studio Code interface with a project named 'LARACRUD'. The Explorer sidebar on the left shows the file structure, with 'User.php' selected under the 'Models' directory. The Terminal panel on the right shows the execution of the command `php artisan breeze:install`. The terminal output displays a series of prompts for selecting a Breeze stack, enabling dark mode, and choosing a testing framework. After selecting 'blade', 'no' for dark mode, and 'PHPUnit', the terminal proceeds to install and build Node dependencies. It shows the installation of 111 packages and auditing of 112 packages. The build process is then initiated, showing the transformation of 47 modules and the rendering of chunks. The final output shows the built files and their sizes, indicating a successful installation of Breeze scaffolding.

```
PS C:\xampp\htdocs\laracrud> php artisan breeze:install

Which Breeze stack would you like to install?
Blade with Alpine .....
React with Inertia .....
Vue with Inertia .....
API only .....
> blade

Would you like dark mode support? (yes/no) [no]
>

Which testing framework do you prefer? [PHPUnit]
PHPUnit .....
Pest .....
>

[INFO] Installing and building Node dependencies.

added 111 packages, and audited 112 packages in 28s

21 packages are looking for funding
  run `npm fund` for details

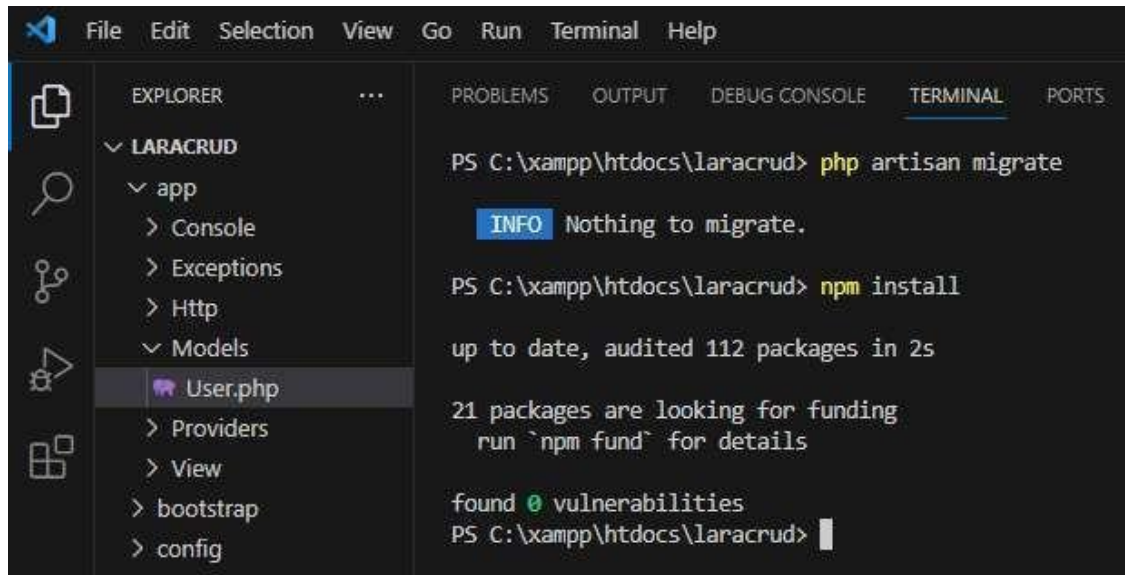
found 0 vulnerabilities

> build
> vite build

vite v4.4.9 building for production...
transforming...
✓ 47 modules transformed.
rendering chunks...
computing gzip size...
public/build/manifest.json 0.26 kB | gzip: 0.13 kB
public/build/assets/app-f88a056c.css 30.13 kB | gzip: 5.85 kB
public/build/assets/app-6e0eadfb.js 71.31 kB | gzip: 26.54 kB
✓ built in 1.99s

[INFO] Breeze scaffolding installed successfully.

PS C:\xampp\htdocs\laracrud>
```



VS Code interface showing the Explorer sidebar with the file structure of a Laravel project. The file 'User.php' is selected under the 'Models' directory. The Terminal panel shows the following commands and output:

```
PS C:\xampp\htdocs\laracrud> php artisan migrate
INFO Nothing to migrate.

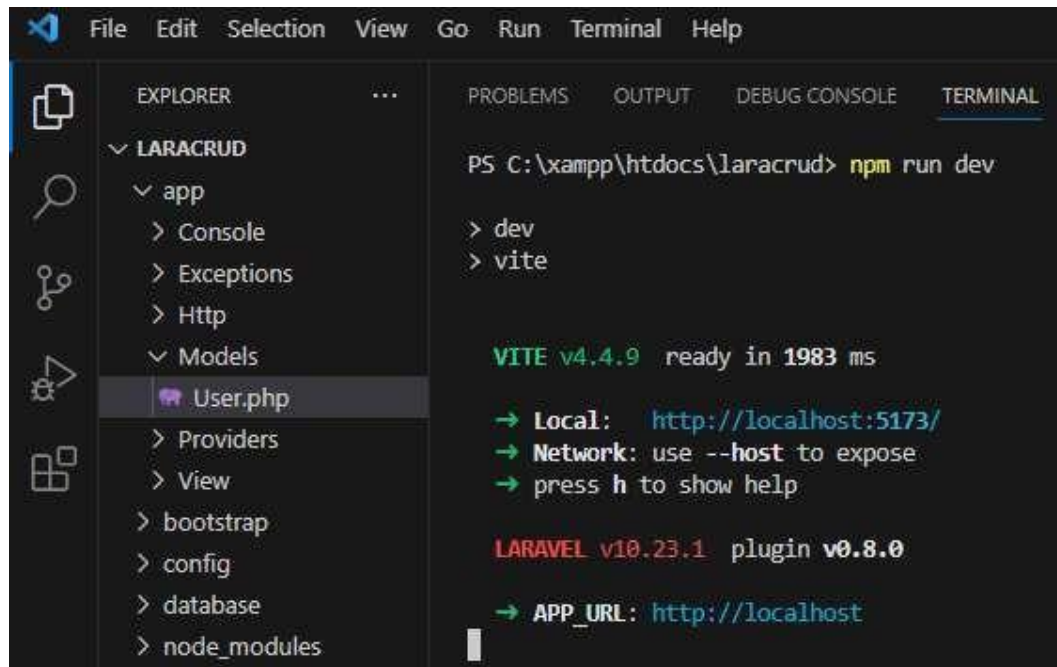
PS C:\xampp\htdocs\laracrud> npm install

up to date, audited 112 packages in 2s

21 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\xampp\htdocs\laracrud>
```

Run the package.



VS Code interface showing the Explorer sidebar with the file structure of a Laravel project. The file 'User.php' is selected under the 'Models' directory. The Terminal panel shows the following commands and output:

```
PS C:\xampp\htdocs\laracrud> npm run dev

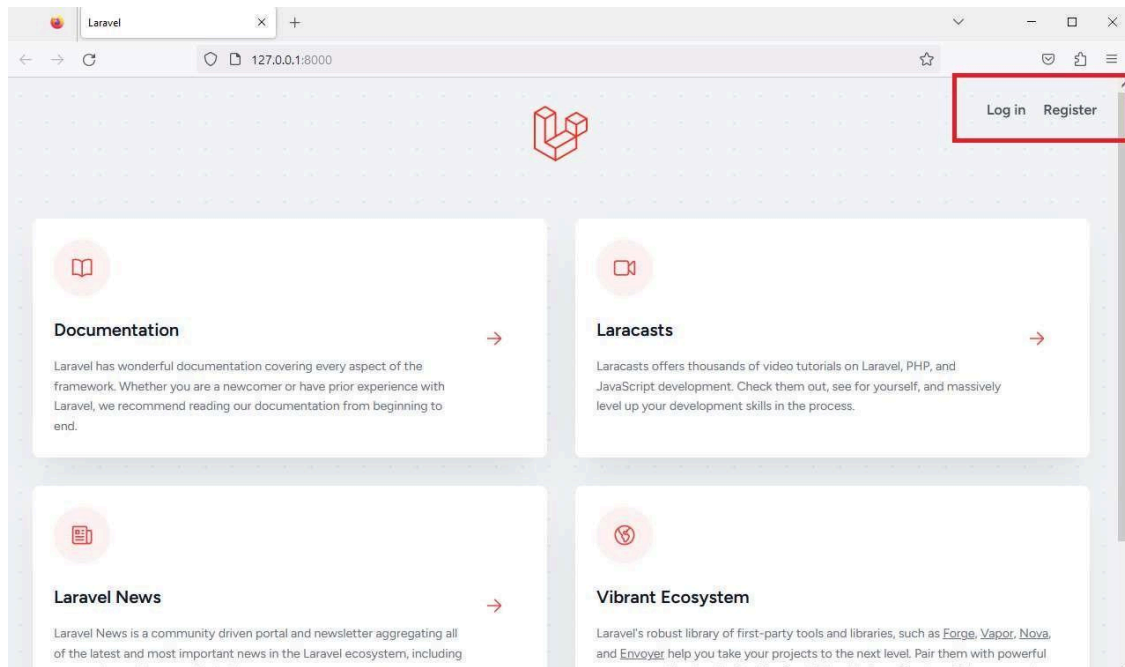
> dev
> vite

VITE v4.4.9 ready in 1983 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h to show help

LARAVEL v10.23.1 plugin v0.8.0

→ APP_URL: http://localhost
```

View page with authentication module



Register



A registration form with the following fields and elements:

- Name:** A text input field containing the text 'LING YU QIAN'.
- Email:** A text input field containing the text 'yqling27@gmail.com'.
- Password:** A password input field with masked characters '.....'.
- Confirm Password:** A text input field with masked characters '.....' and a cursor at the end.
- Buttons:** A link labeled 'Already registered?' and a dark blue button labeled 'REGISTER'.

Logged in



Dashboard

CRUD

LING YU QIAN ▾

Dashboard

You're logged in!

4.0 CRUD PAGE

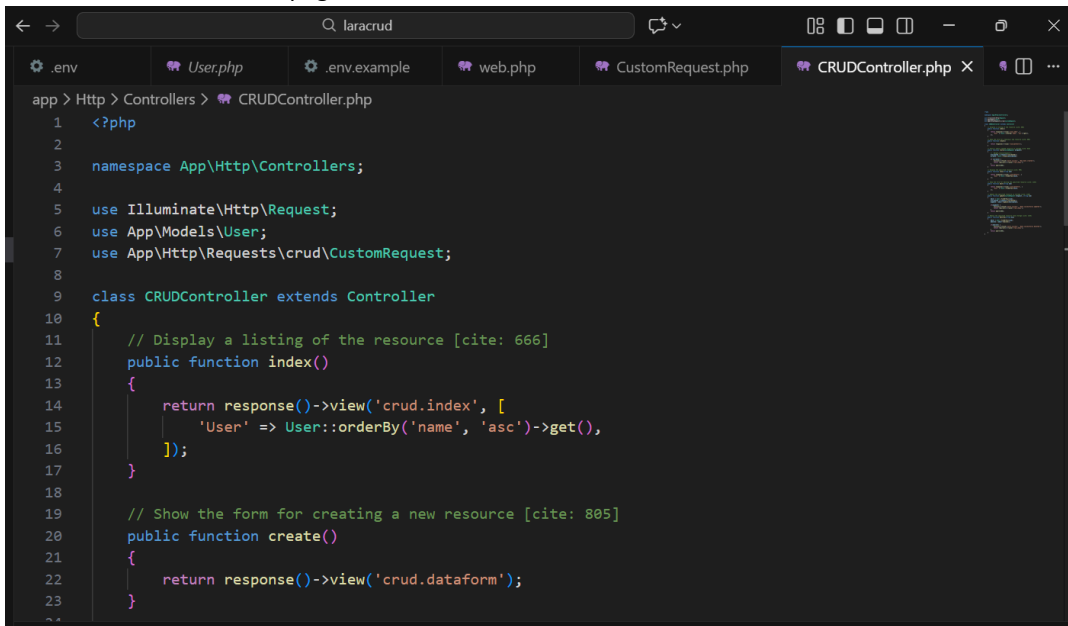
Add CRUDController path and route at web.php, then create the controller using terminal.

```
routes > web.php
1  <?php
2
3  use App\Http\Controllers\ProfileController;
4  use App\Http\Controllers\CRUDController; // Import the controller
5  use Illuminate\Support\Facades\Route;
6
7  Route::get('/', function () {
8      return view('welcome');
9  });
10
11 Route::get('/dashboard', function () {
12     return view('dashboard');
13 })->middleware(['auth', 'verified'])->name('dashboard');
14
15 Route::middleware('auth')->group(function () {
16     Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
17     Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
18     Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
19
20     // Add CRUD Route
21     Route::resource('crud', CRUDController::class);
22 });
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\xampp\htdocs\laracrud> php artisan make:controller CRUDController --resource
```

View the CRUDController page.



The screenshot shows a code editor with the following tabs: .env, User.php, .env.example, web.php, CustomRequest.php, and CRUDController.php. The active tab is CRUDController.php, which contains the following PHP code:

```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6 use App\Models\User;
7 use App\Http\Requests\crud\CustomRequest;
8
9 class CRUDController extends Controller
10 {
11     // Display a listing of the resource [cite: 666]
12     public function index()
13     {
14         return response()->view('crud.index', [
15             'User' => User::orderBy('name', 'asc')->get(),
16         ]);
17     }
18
19     // Show the form for creating a new resource [cite: 805]
20     public function create()
21     {
22         return response()->view('crud.dataform');
23     }
24 }
```

Add new menu at navigation.blade.php and SAVE (Copy from training material: 01_nav.txt).

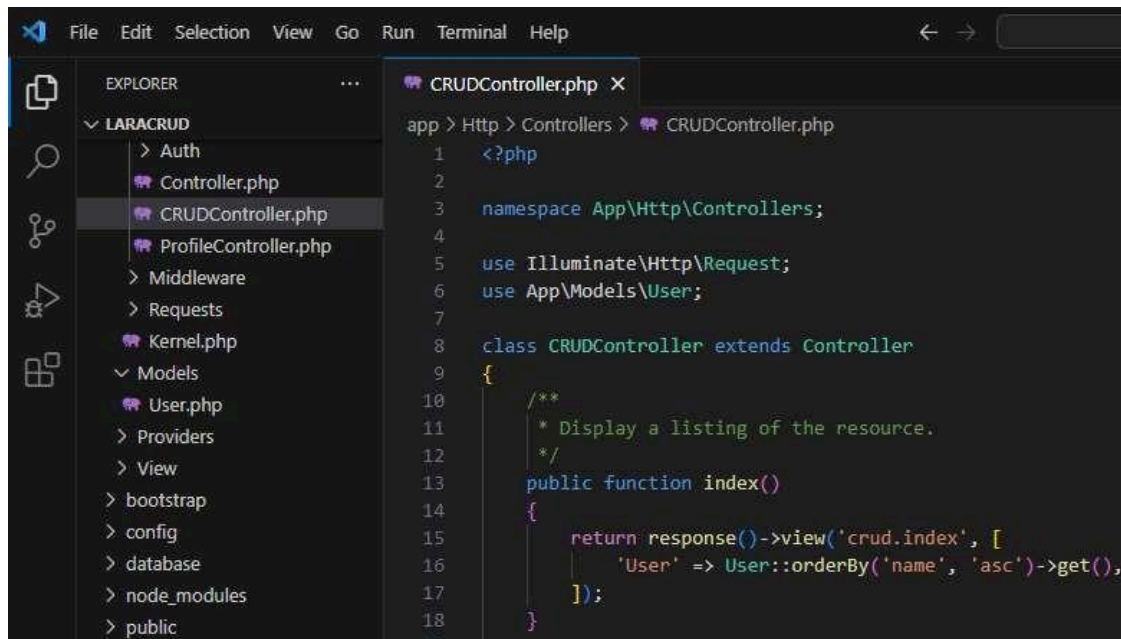


The screenshot shows a code editor with the following code in the navigation.blade.php file:

```
<!-- Navigation Links -->
<div class="hidden space-x-8 sm:-my-px sm:ml-10 sm:flex">
    <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
        {{ __('Dashboard') }}
    </x-nav-link>

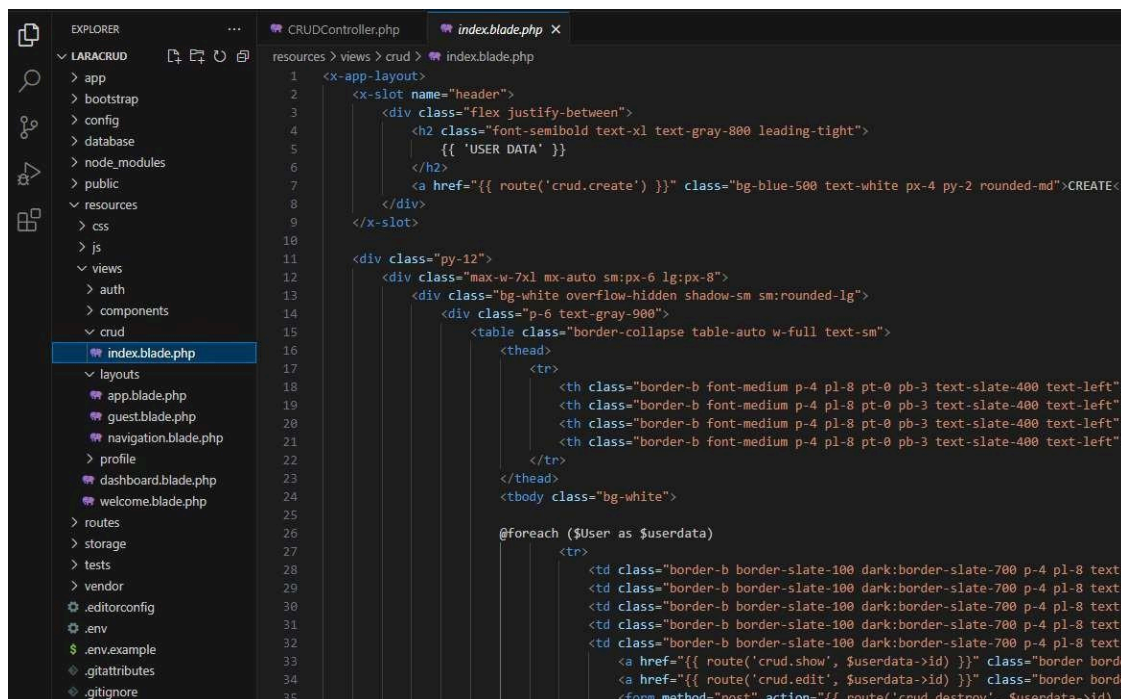
    <x-nav-link :href="route('crud.index')" :active="request()->routeIs('crud.index')">
        {{ 'CRUD' }}
    </x-nav-link>
```

Add User model and index function syntax at CRUDController to view and sort all data.



```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use Illuminate\Http\Request;
6 use App\Models\User;
7
8 class CRUDController extends Controller
9 {
10     /**
11      * Display a listing of the resource.
12      */
13     public function index()
14     {
15         return response()->view('crud.index', [
16             'User' => User::orderBy('name', 'asc')->get(),
17         ]);
18     }
19 }
```

Create crud folder in view and create new blade file index.blade.php. Write code to display all data. (Copy from training material: 02_list.txt).



```
1 <x-app-layout>
2     <x-slot name="header">
3         <div class="flex justify-between">
4             <h2 class="font-semibold text-xl text-gray-800 leading-tight">
5                 {{ 'USER DATA' }}
6             </h2>
7             <a href="{{ route('crud.create') }}" class="bg-blue-500 text-white px-4 py-2 rounded-md">CREATE</a>
8         </div>
9     </x-slot>
10
11     <div class="py-12">
12         <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
13             <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
14                 <div class="p-6 text-gray-900">
15                     <table class="border-collapse table-auto w-full text-sm">
16                         <thead>
17                             <tr>
18                                 <th class="border-b font-medium p-4 pl-8 pt-0 pb-3 text-slate-400 text-left">
19                                     <th class="border-b font-medium p-4 pl-8 pt-0 pb-3 text-slate-400 text-left">
20                                     <th class="border-b font-medium p-4 pl-8 pt-0 pb-3 text-slate-400 text-left">
21                                     <th class="border-b font-medium p-4 pl-8 pt-0 pb-3 text-slate-400 text-left">
22                             </tr>
23                         </thead>
24                         <tbody class="bg-white">
25                             @foreach ($User as $userdata)
26                                 <tr>
27                                     <td class="border-b border-slate-100 dark:border-slate-700 p-4 pl-8 text-
28                                     <td class="border-b border-slate-100 dark:border-slate-700 p-4 pl-8 text-
29                                     <td class="border-b border-slate-100 dark:border-slate-700 p-4 pl-8 text-
30                                     <td class="border-b border-slate-100 dark:border-slate-700 p-4 pl-8 text-
31                                     <td class="border-b border-slate-100 dark:border-slate-700 p-4 pl-8 text-
32                                     <a href="{{ route('crud.show', $userdata->id) }}" class="border borde
33                                     <a href="{{ route('crud.edit', $userdata->id) }}" class="border borde
34                                     <form method="post" action="{{ route('crud.destroy', $userdata->id) }}
35                                 </tr>
36                             </tbody>
37                         </table>
38                     </div>
39                 </div>
40             </div>
41         </div>
42     </div>
43 </x-app-layout>
```


SAVE, restart Laravel server and breeze package.

```
PS C:\xampp\htdocs\laracrud> php artisan serve

INFO Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

PS C:\xampp\htdocs\laracrud> npm run dev

> dev
> vite

VITE v4.4.9 ready in 426 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h to show help

LARAVEL v10.23.1 plugin v0.8.0

→ APP_URL: http://localhost
```

View the index CRUD page.

Dashboard

CRUD

LING YU QIAN

USER DATA

CREATE

Student ID	Name	Email	Register Date	Action
9	ling admin	yuqian@gmail.com	2025-12-30 02:00:35	DETAILSUPDATEDELETE
8	ling yu qian	000yuqian@gmail.com	2025-12-30 01:57:31	DETAILSUPDATEDELETE
10	LING YU QIAN	yqling27@gmail.com	2026-01-23 15:15:17	DETAILSUPDATEDELETE
5	lingyuqian	ling@gmail.com	2025-12-24 15:52:37	DETAILSUPDATEDELETE

5.0 CREATE

Add syntax to view webform and store data with validation.

```
CRUDController.php X
app > Http > Controllers > CRUDController.php
19
20 /**
21  * Show the form for creating a new resource.
22  */
23 public function create()
24 {
25     return response()->view('crud.dataform');
26 }
27
28 /**
29  * Store a newly created resource in storage.
30  */
31 public function store(CustomRequest $request)
32 {
33     $validated = $request->validated();
34     $create = User::create($validated);
35     if($create) {
36         session()->flash('notif.success', 'New data created');
37         return redirect()->route('crud.index');
38     }
39     return abort(500); //500 server error
40 }
```

Create CustomRequest file.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\xampp\htdocs\laracrud> php artisan make:request crud/CustomRequest

INFO Request [C:\xampp\htdocs\laracrud\app\Http\Requests/crud/CustomRequest.php] created successfully.
PS C:\xampp\htdocs\laracrud>
```

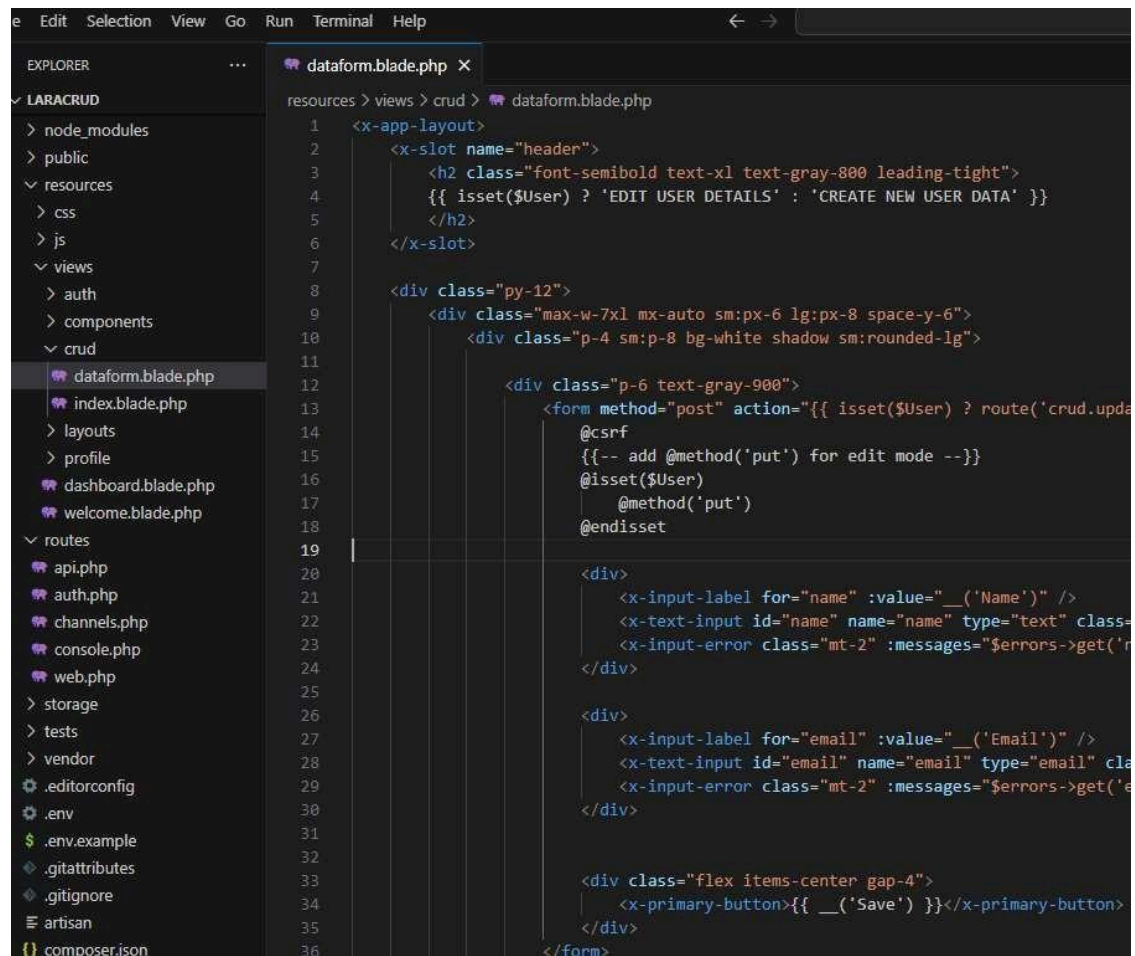

Add CustomRequest path.

```
CRUDController.php X
app > Http > Controllers > CRUDController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\User;
7  use App\Http\Requests\crud\CustomRequest;
```

Change return true and add custom validation rules at CustomRequest.php.

```
CRUDController.php  CustomRequest.php X
app > Http > Requests > crud > CustomRequest.php
1  <?php
2
3  namespace App\Http\Requests\crud;
4
5  use Illuminate\Foundation\Http\FormRequest;
6
7  class CustomRequest extends FormRequest
8  {
9      /**
10       * Determine if the user is authorized to make this request.
11       */
12      public function authorize(): bool
13      {
14          return true;
15      }
16
17      /**
18       * Get the validation rules that apply to the request.
19       *
20       * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array|string>
21       */
22      public function rules(): array
23      {
24          return [
25              'name' => 'required|string|min:6|max:255',
26              'email' => 'required|string|min:12|max:255',
27          ];
28      }
29  }
```

Add new blade file in crud view folder and write code to create an input form (Copy from training material: 03_form.txt).



```
1 <x-app-layout>
2   <x-slot name="header">
3     <h2 class="font-semibold text-xl text-gray-800 leading-tight">
4       {{ isset($User) ? 'EDIT USER DETAILS' : 'CREATE NEW USER DATA' }}
5     </h2>
6   </x-slot>
7
8   <div class="py-12">
9     <div class="max-w-7xl mx-auto sm:px-6 lg:px-8 space-y-6">
10      <div class="p-4 sm:p-8 bg-white shadow sm:rounded-lg">
11
12        <div class="p-6 text-gray-900">
13          <form method="post" action="{{ isset($User) ? route('crud.update') : route('crud.create') }}">
14            @csrf
15            {{-- add @method('put') for edit mode --}}
16            @isset($User)
17              @method('put')
18            @endisset
19
20
21            <div>
22              <x-input-label for="name" :value="__('Name')"/>
23              <x-text-input id="name" name="name" type="text" class="mt-1">
24            </div>
25
26            <div>
27              <x-input-label for="email" :value="__('Email')"/>
28              <x-text-input id="email" name="email" type="email" class="mt-1">
29              <x-input-error class="mt-2" :messages="$errors->get('email')"/>
30            </div>
31
32            <div class="flex items-center gap-4">
33              <x-primary-button>{{ __('Save') }}</x-primary-button>
34            </div>
35          </form>
36        </div>
37      </div>
38    </div>
39  </div>
40</x-app-layout>
```

Add code to app.blade.php to display operation notifications.

```
app.blade.php X
resources > views > layouts > app.blade.php
20
21 <!-- Page Heading -->
22 @if (isset($header))
23 <header class="bg-white shadow">
24 <div class="max-w-7xl mx-auto py-6 px-4 sm:px-6 lg:px-8">
25 {{ $header }}
26
27 @if (Session::has('notif.success'))
28 <div class="bg-blue-300 mt-2 p-4">
29 <span class="text-white">{{ Session::get('notif.success') }}</span>
30 </div>
31 @endif
32
33 </div>
34 </header>
35 @endif
```

Test custom validation

Name

The name field must be at least 6 characters.

Set password field allow NULL in phpMyAdmin to avoid error. Add new data.

The screenshot shows a web application dashboard with a top navigation bar containing a logo, 'Dashboard', and 'CRUD' (which is underlined). On the right of the navigation bar is the user name 'LING YU QIAN' with a dropdown arrow. Below the navigation bar, the page title 'USER DATA' is displayed on the left, and a blue 'CREATE' button is on the right. A blue notification banner at the top of the main content area says 'New data created'. The main content area features a table with the following data:

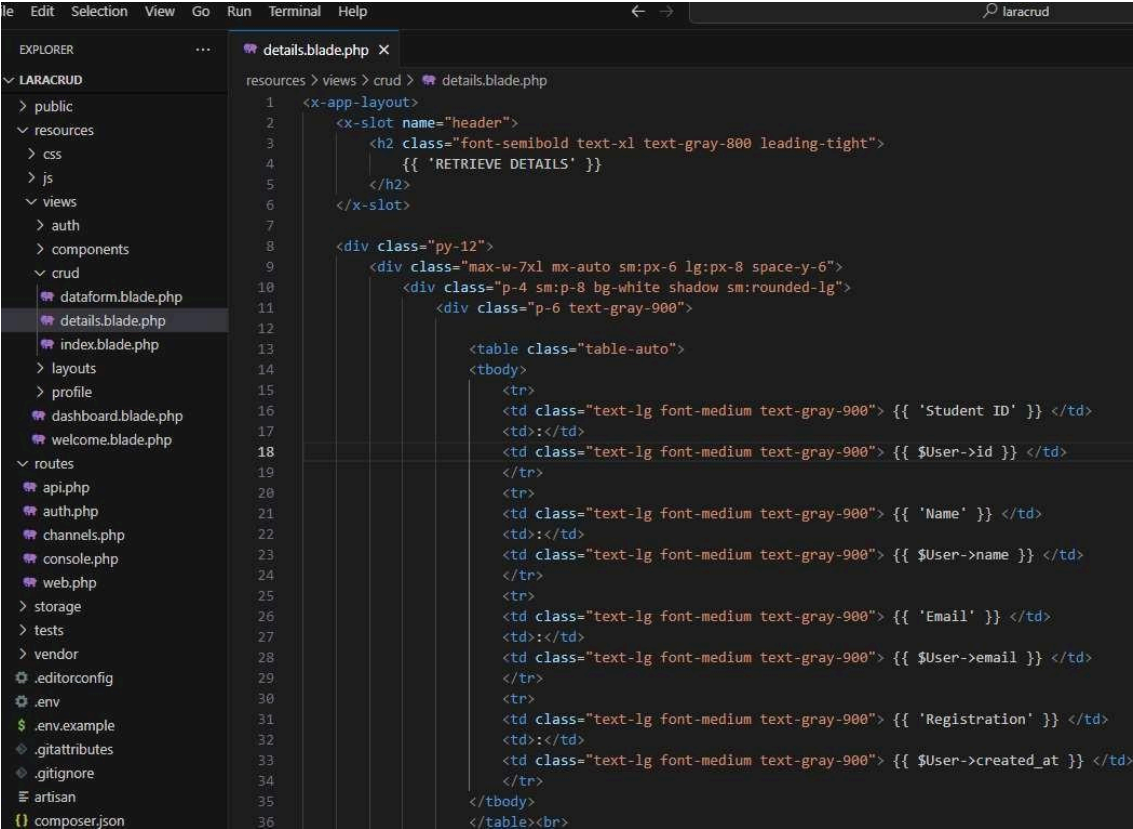
Student ID	Name	Email	Register Date	Action
9	ling admin	yuqian@gmail.com	2025-12-30 02:00:35	DETAILS UPDATE DELETE
8	ling yu qian	000yuqian@gmail.com	2025-12-30 01:57:31	DETAILS UPDATE DELETE
10	LING YU QIAN	yqling27@gmail.com	2026-01-23 15:15:17	DETAILS UPDATE DELETE
5	lingyuqian	ling@gmail.com	2025-12-24 15:52:37	DETAILS UPDATE DELETE

6.0 RETRIEVE

Add open view code at CRUDController.

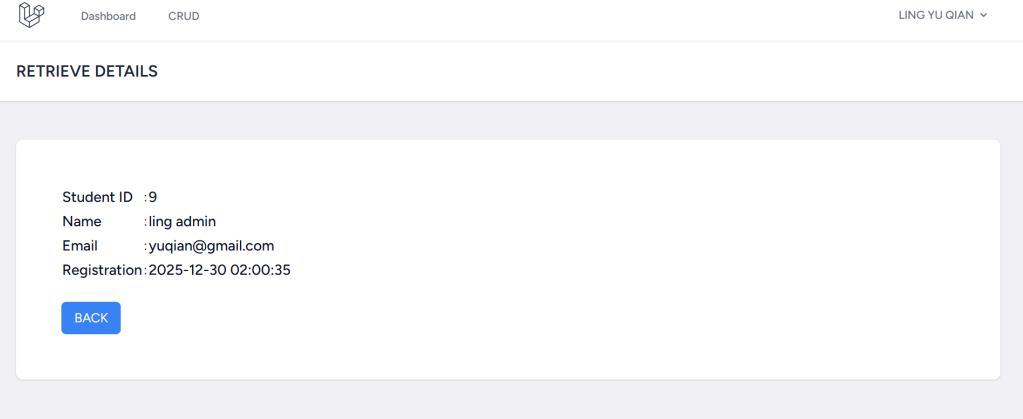
```
CRUDController.php
app > Http > Controllers > CRUDController.php
42
43  /**
44   * Display the specified resource.
45   */
46  public function show(string $id)
47  {
48      return response()->view('crud.details', [
49          'User' => User::findOrFail($id),
50      ]);
51  }
```

Add new blade file in crud view folder and write code to display retrieved data (Copy from training material: 04_view.txt).



```
1 <x-app-layout>
2   <x-slot name="header">
3     <h2 class="font-semibold text-xl text-gray-800 leading-tight">
4       {{ 'RETRIEVE DETAILS' }}
5     </h2>
6   </x-slot>
7
8   <div class="py-12">
9     <div class="max-w-7xl mx-auto sm:px-6 lg:px-8 space-y-6">
10      <div class="p-4 sm:p-8 bg-white shadow sm:rounded-lg">
11        <div class="p-6 text-gray-900">
12
13          <table class="table-auto">
14            <tbody>
15              <tr>
16                <td class="text-lg font-medium text-gray-900"> {{ 'Student ID' }} </td>
17                <td></td>
18              </tr>
19              <tr>
20                <td class="text-lg font-medium text-gray-900"> {{ $User->id }} </td>
21                <td></td>
22              </tr>
23              <tr>
24                <td class="text-lg font-medium text-gray-900"> {{ 'Name' }} </td>
25                <td></td>
26              </tr>
27              <tr>
28                <td class="text-lg font-medium text-gray-900"> {{ $User->name }} </td>
29                <td></td>
30              </tr>
31              <tr>
32                <td class="text-lg font-medium text-gray-900"> {{ 'Email' }} </td>
33                <td></td>
34              </tr>
35              <tr>
36                <td class="text-lg font-medium text-gray-900"> {{ $User->email }} </td>
37                <td></td>
38              </tr>
39              <tr>
40                <td class="text-lg font-medium text-gray-900"> {{ 'Registration' }} </td>
41                <td></td>
42              </tr>
43              <tr>
44                <td class="text-lg font-medium text-gray-900"> {{ $User->created_at }} </td>
45                <td></td>
46              </tr>
47            </tbody>
48          </table><br>
49        </div>
50      </div>
51    </div>
52  </div>
53</x-app-layout>
```

Click DETAILS button to view detail data.



7.0 UPDATE

Write code to view same dataform and update data with custom validation.

```
CRUDController.php
app > Http > Controllers > CRUDController.php
53  /**
54   * Show the form for editing the specified resource.
55   */
56  public function edit(string $id)
57  {
58      return response()->view('crud.dataform', [
59          'User' => User::findOrFail($id),
60      ]);
61  }
62
63  /**
64   * Update the specified resource in storage.
65   */
66  public function update(CustomRequest $request, string $id)
67  {
68      $User = User::findOrFail($id);
69      $validated = $request->validated();
70      $update = $User->update($validated);
71      if($update) {
72          session()->flash('notif.success', 'Data successfully updated');
73          return redirect()->route('crud.index');
74      }
75      return abort(500);
76  }
```

Test the custom validation on update page.

The name field must be at least 6 characters.

Update data (Do not update email credentials for logged in user)

USER DATA

CREATE

Data successfully updated

Student ID	Name	Email	Register Date	Action
9	ling admin utm	yuqianutm@gmail.com	2025-12-30 02:00:35	DETAILSUPDATEDELETE

8.0 DELETE

Add code to delete data

```
CRUDController.php X
app > Http > Controllers > CRUDController.php
78      /**
79      * Remove the specified resource from storage.
80      */
81      public function destroy(string $id)
82      {
83          $User = User::findOrFail($id);
84          $delete = $User->delete($id);
85          if($delete) {
86              session()->flash('notif.success', 'Data successfully deleted');
87              return redirect()->route('crud.index');
88          }
89          return abort(500);
90      }
91  }
```

Delete data.

 Dashboard **CRUD** ling admin utm ▾

USER DATA [CREATE](#)

Data successfully deleted

Student ID	Name	Email	Register Date	Action
9	ling admin utm	yuqianutm@gmail.com	2025-12-30 02:00:35	DETAILS UPDATE DELETE
8	ling yu qian	000yuqian@gmail.com	2025-12-30 01:57:31	DETAILS UPDATE DELETE
5	lingyuqian	ling@gmail.com	2025-12-24 15:52:37	DETAILS UPDATE DELETE

Check the deleted data in phpMyAdmin.

Server: 127.0.0.1 » Database: cruddata » Table: users

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Operations

Tracking

Triggers

Showing rows 0 - 2 (3 total, Query took 0.0005 seconds.)

SELECT * FROM `users`

Profiling

[Edit inline]

[Edit]

[Explain SQL]

[Create PHP code]

[Refresh]

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

+T→

id

name

email

email_verified_at

password

remember_token

created_at

updated_at

Edit

Copy

Delete

5

lingyuqian

ling@gmail.com

NULL

NULL

NULL

2025-12-24 15:52:37

2025-12-24 15:52:37

Edit

Copy

Delete

8

ling yu qian

000yuqian@gmail.com

NULL

NULL

NULL

2025-12-30 01:57:31

2025-12-30 01:57:31

Edit

Copy

Delete

9

ling admin utm

yuqianutm@gmail.com

NULL

\$2y\$12\$DHx.tpqljZ4oHxzYqyEFeGFCQtIG7GQFVSME03VINr...

NULL

2025-12-30 02:00:35

2026-01-23 15:27:04